Statsmodels

What is statsmodels

Statsmodels is a Python module that helps to implement and test statistical models, test hypotheses and explore data. It has tested its results against existing statistical packages and softwares, such as R, Stata, and SAS, to ensure correctness. Statsmodels is released under an open source Modified BSD (3-clause) license.

Statsmodels combines the use of other statistical softwares: it uses formulas in an R-Studio format, and dataframes from pandas.

Note: Statsmodels was tested and released as a new package during Google Summer of Code 2009, and has been under constant development since then.

Main features of statsmodels

1. Linear Models
2. Generalised linear models
3. Linear mixed effects models
4. Generalised estimating equations
5. Discrete models
6. Time series analysis
7. Multivariate

Dependencies in statsmodels

Current direct dependencies:

- Python >= 2.5
- Numpy >= 1.11
- Scipy >= 0.18
- Pandas >= 0.19
- Cython >= 0.24
- Patsy >= 0.4.0

To build statsmodels, the user must also have a C compiler installed because of the package's dependency on Cython, as well as a suitable Python installation. The user should note that Cython is only needed if we build the code from the github repository. If the code is being built from a source distribution, Cython dependency is not needed.

Dependencies in statsmodels

Current indirect dependencies:

- Matplotlib >= 1.5 (plotting functions, running pre-installed examples)
- X-12-ARIMA, X-13-ARIMA-SEATS (time-series analysis)
- Pytest (run test suite)
- IPython >= 3.0 (build documents locally, use notebooks)
- Joblib >= 0.9 (to accelerate distributed estimation for some models)
- Jupyter (use notebooks)

ARIMA on housing prices

Evaluating statsmodels in Python3 on a real-life application

The Problem

Aim: provide insight to investors interested in growing their finances through real estate.

Constraints: budget under USD 500K, buy in 2020.

Area of interest: Chicago with average house price under USD 3 million in 2016-18.

Main result: MSE on dynamic forecast

What is ARIMA?

Autoregressive Integrated Moving Average

1. Use the statsmodels library to fit an ARIMA model.
2. Pass in p, d, q parameters.
    a. ARIMA(p,q,d): p is the order of the autoregressive model, q is the order of the MA model, d is the degree of differencing.
3. Prepare the model on the training data using the fit() function.
4. Call predict() function and specify the index (or indices) of the time(s) to be predicted.

ARIMA in Python3 using statsmodels

1. Import necessary libraries and dataset

In [50]:
```
# statsmodels for modeling the data later
import statsmodels.api as sm
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.seasonal import seasonal_decompose as sd
from sklearn.metrics import mean_squared_error
from sklearn.linear_model import LassoLarsCV
```

In [128]:
```
df = pd.read_csv('zillow_data.csv')
df.head()
```

Out[128]:

Metro	CountyName	SizeRank	1996-04	1996-05	1996-06	...	2017-07	2017-08	2017-09	2017-10	2017-11	2017-12	2018-01	2018-02	2018-03	2018-04
icago	Cook	1	334200.0	335400.0	336500.0	...	1005500	1007500	1007800	1009600	1013300	1018700	1024400	1030700	1033800	1030600
allas-Fort Worth	Collin	2	235700.0	236900.0	236700.0	...	308000	310000	312500	314100	315000	316600	318100	319600	321100	321800
uston	Harris	3	210400.0	212200.0	212200.0	...	321000	320600	320200	320400	320800	321200	321200	323000	326900	329900
icago	Cook	4	498100.0	500900.0	503100.0	...	1289800	1287700	1287400	1291500	1296600	1299000	1302700	1306400	1308500	1307000
Paso	El Paso	5	77300.0	77300.0	77300.0	...	119100	119400	120000	120300	120300	120300	120300	120500	121000	121500

2. Reshape data for processing

In [132]:
```
#2. Reshaping the data

def melt_data(df):
    melted = pd.melt(df, id_vars=['RegionName'], var_name='Month', value_name = 'MeanValue')
    melted['Month'] = pd.to_datetime(melted['Month'], format = '%Y-%m')
    melted = melted.dropna(subset=['MeanValue'])
    return melted
```

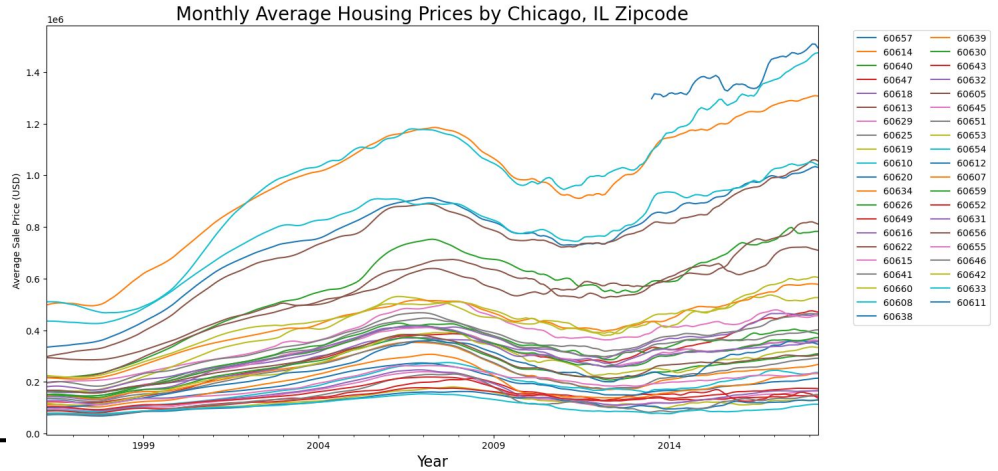
In [153]:
```
dfm = melt_data(chi1)
dfm.head()
```

Out[153]:

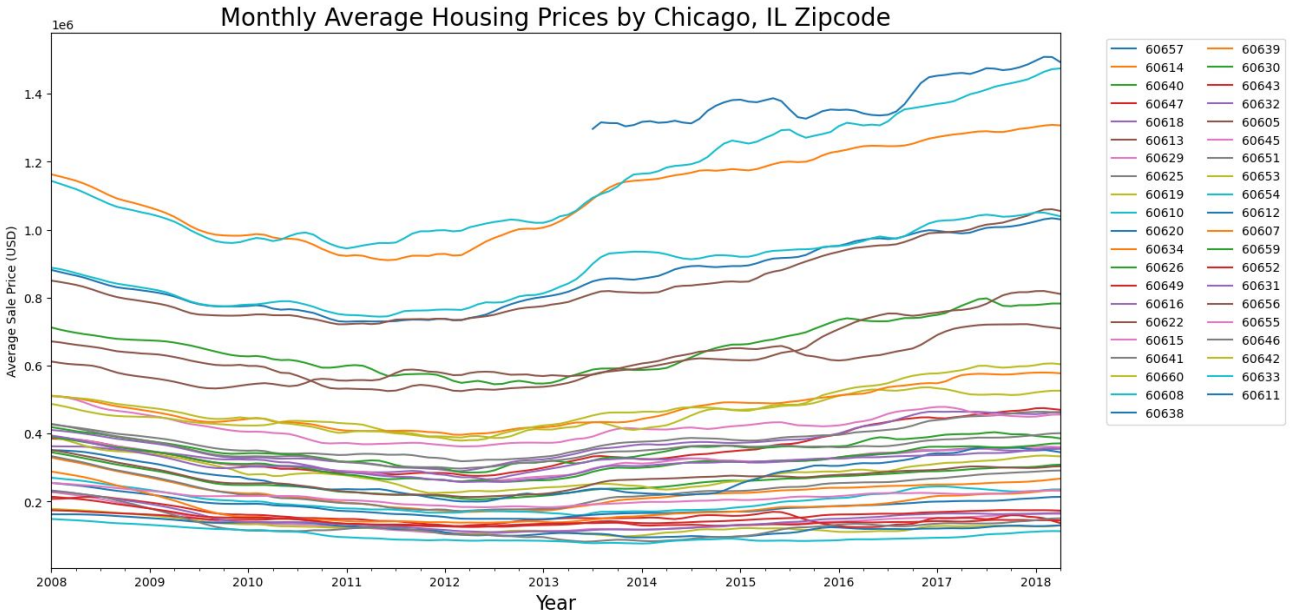
	RegionName	Month	MeanValue
0	60657	1996-04-01	334200.0
1	60614	1996-04-01	498100.0
2	60640	1996-04-01	216500.0
3	60647	1996-04-01	122700.0
4	60618	1996-04-01	142600.0

3. EDA and Visualisation

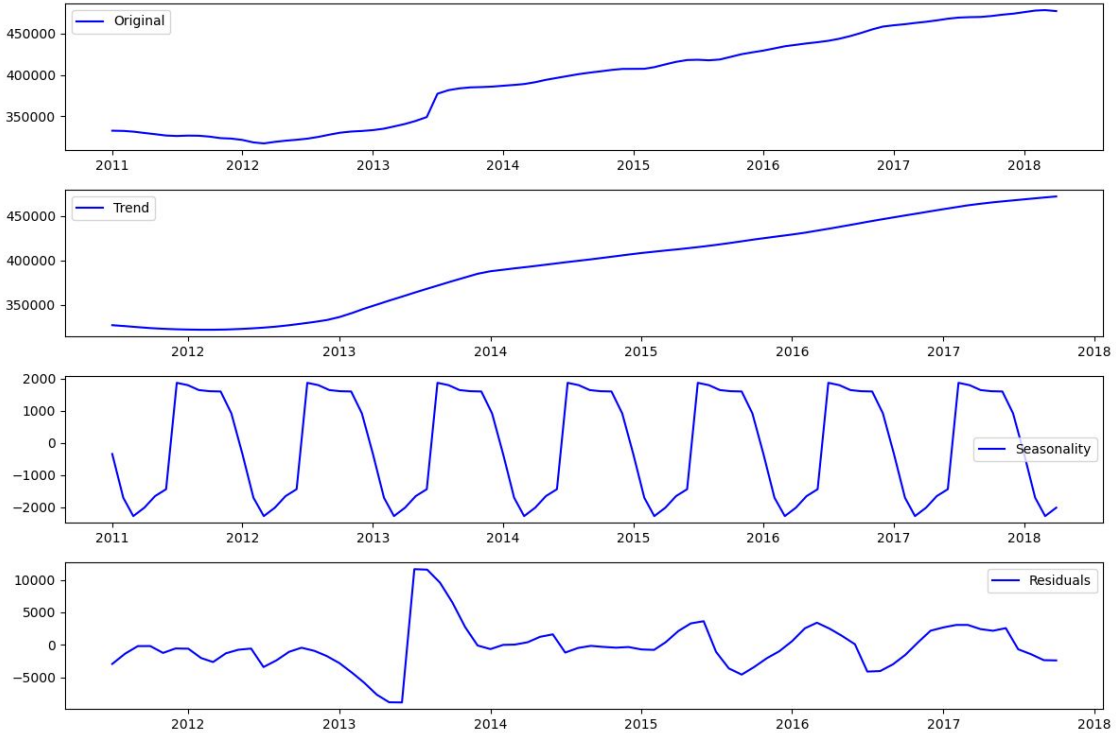
In [138]:
```
#3. EDA and visualisation

for zipcode in dfm.RegionName.unique():
    temp_df = dfm[dfm.RegionName == zipcode]
    temp_df['MeanValue'].plot(figsize = (15,8), label=zipcode)

plt.legend(bbox_to_anchor=(1.04,1), loc='upper left', ncol=2)
plt.xlabel("Year", fontsize = 16)
plt.ylabel("Average Sale Price (USD)")
plt.title('Monthly Average Housing Prices by Chicago, IL Zipcode', fontsize = 20);
```



In [139]:
```
# A deeper look at 2008 above
for zipcode in dfm.RegionName.unique():
    temp_df = dfm[dfm.RegionName == zipcode]
    temp_df['2008':]['MeanValue'].plot(figsize = (15,8), label=zipcode)

plt.legend(bbox_to_anchor=(1.04,1), loc='upper left', ncol=2)
plt.xlabel("Year", fontsize = 16)
plt.ylabel("Average Sale Price (USD)")
plt.title('Monthly Average Housing Prices by Chicago, IL Zipcode', fontsize = 20);
```

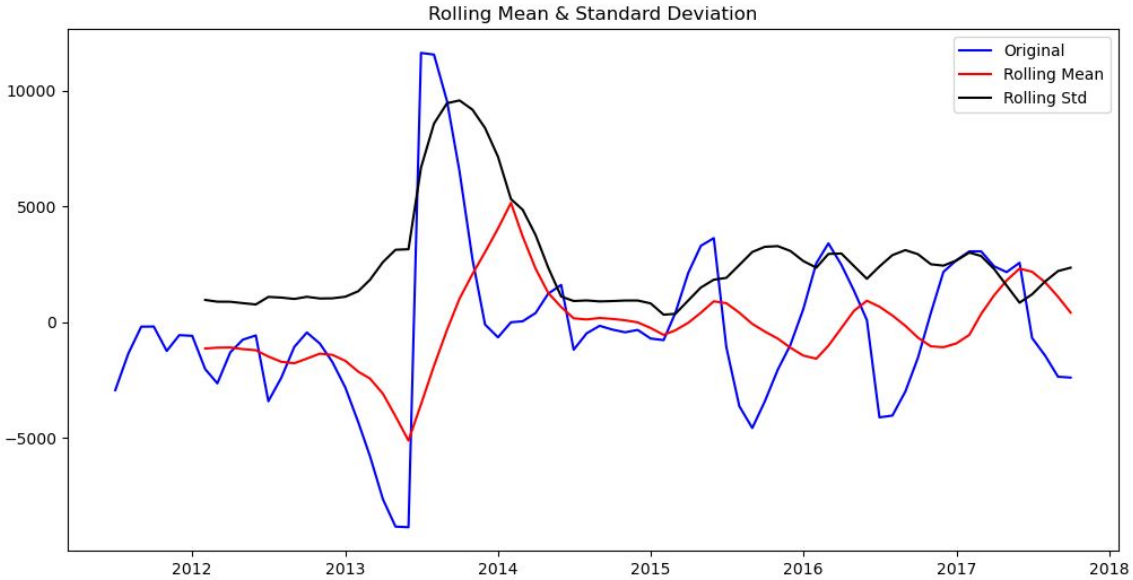


Seasonal decomp 2011-now



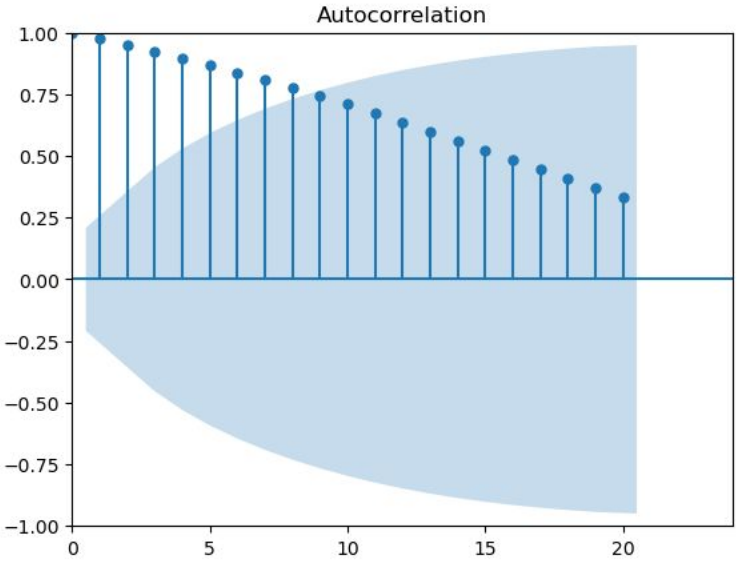
Stationarity test

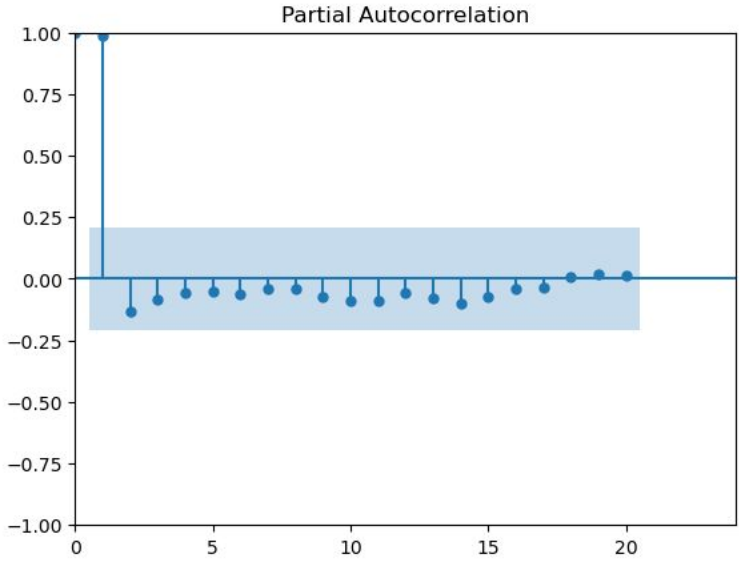
```
Results of Dickey-Fuller Test:

Test Statistic                  -4.935243
p-value                          0.000030
#Lags Used                       2.000000
Number of Observations Used     73.000000
Critical Value (1%)             -3.523284
Critical Value (5%)             -2.902031
Critical Value (10%)            -2.588371
dtype: float64
```



Data exhibits stationarity

Autocorrelation test





4. SARIMA on all zip codes

In [74]:
```
# Define the p, d and q parameters to take any value between 0 and 2
p = d = q = range(0,2)
# Generate all different combinations of p, d and q triplets
pdq = list(itertools.product(p,d,q))
# Generate all different combinations of seasonal p, d and q triplets
pdqs = [(x[0], x[1], x[2], 12) for x in list(itertools.product(p, d, q))]
```

In [75]:
```
#Run SARIMA
start=time.time()
ans = []

for df, name in zip(zip_dfs, zip_list):
    for para1 in pdq:
        for para2 in pdqs:
            try:
                mod = sm.tsa.statespace.SARIMAX(df,
                                                order = para1,
                                                seasonal_order = para2,
                                                enforce_stationarity = False,
                                                enforce_invertibility = False)
                output = mod.fit()
                ans.append([name, para1, para2, output.aic])
                print('Result for {}'.format(name) + ' ARIMA {} x {}12 : AIC Calculated = {}'.format(para1, para2, o
            except:
                continue
```

In [76]:
```
print('Took', time.time()-start, 'seconds.')
```

```
Took 337.8995158672333 seconds.
```

In [77]:
```
result = pd.DataFrame(ans, columns = ['name','pdq','pdqs','AIC'])
```

In [79]:
```
best_para = result.loc[result.groupby("name")["AIC"].idxmin()]
```

In [80]:
```
best_para.head()
```

Out[80]:

	name	pdq	pdqs	AIC
1663	60605	(1, 1, 1)	(1, 1, 1, 12)	4395.621359
2043	60607	(1, 1, 1)	(0, 1, 1, 12)	4135.226203
1279	60608	(1, 1, 1)	(1, 1, 1, 12)	3890.849597
639	60610	(1, 1, 1)	(1, 1, 1, 12)	4423.624017
2622	60611	(1, 1, 1)	(1, 1, 0, 12)	677.089829

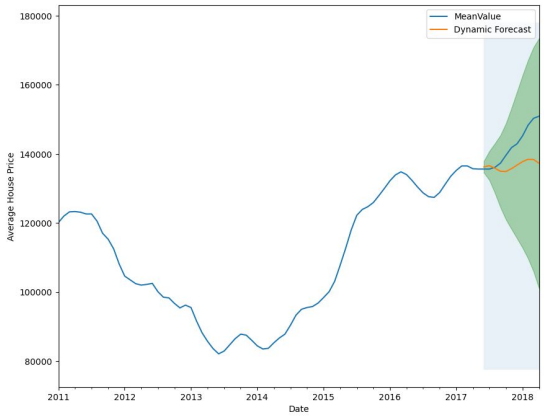
In [82]:
```
# Make dynamic forecast on 2017-06-01 onwards and compare with real values

#Make Prediction and compare with real values
summary_table = pd.DataFrame()
Zipcode = []
MSE_Value = []
models = []
for name, pdq, pdqs, df in zip(best_para['name'], best_para['pdq'], best_para['pdqs'], zip_dfs):


    ARIMA_MODEL = sm.tsa.SARIMAX(df,
                                 order = pdq,
                                 seasonal_order = pdqs,
                                 enforce_stationarity = False,
                                 enforce_invertibility = False,
                                 )


    output = ARIMA_MODEL.fit()
    models.append(output)

    #get dynamic predictions starting 2017-06-01
    pred_dynamic = output.get_prediction(start=pd.to_datetime('2017-06-01'), dynamic = True, full_results = True)
    pred_dynamic_conf = pred_dynamic.conf_int()
    zip_forecasted = pred_dynamic.predicted_mean
    zip_truth = df['2017-06-01':]['MeanValue']

    sqrt_mse = np.sqrt(((zip_forecasted - zip_truth)**2).mean())

    Zipcode.append(name)
    MSE_Value.append(sqrt_mse)

summary_table['Zipcode'] = Zipcode
summary_table['Sqrt_MSE'] = MSE_Value
```

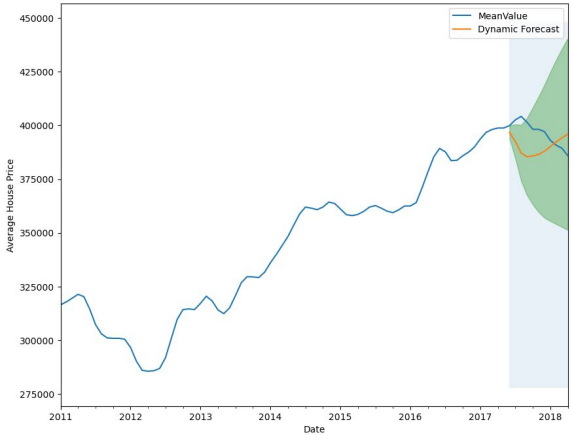
MSEs on a smaller subset

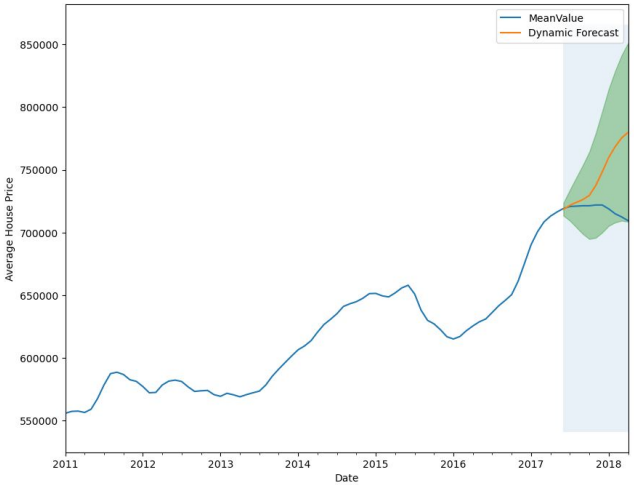
```
In [127]: summary_table
```

Out[127]:

	Zipcode	Sqrt_MSE
0	60605	7297.329880
1	60626	10295.067400
2	60651	36323.879665







Interpretation

1. The ARIMA model tries to predict the housing prices. The pink line is our prediction, the blue our expected values.

2. The MSE is pretty high, which means our dataset is used to predict future values with poor accuracy.

   MSE closer to 0 is better (estimator is predicting with accuracy).

   However, we used dynamic forecast. A higher MSE is expected since we are relying on fewer historical data.

Conclusion

Why choose ARIMA?

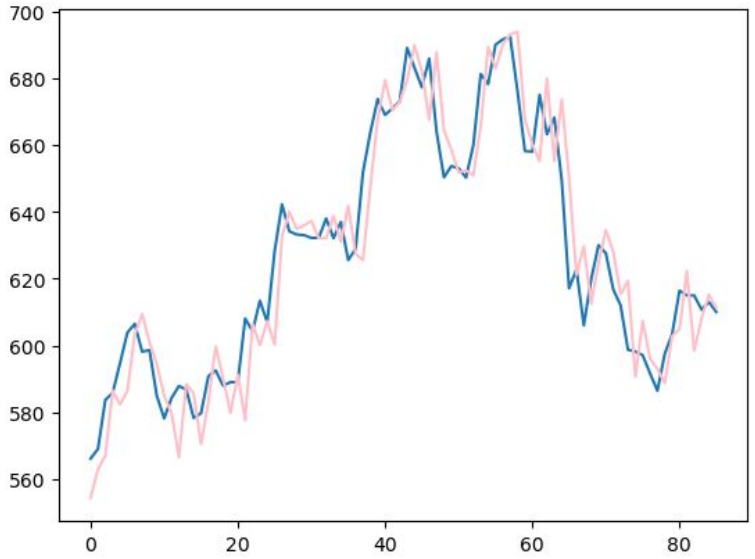
1. Forecasts **several** things (demand, stocks, etc)
2. Uses differencing to **convert non-stationary** to stationary for prediction **using historical data**
3. Performs well on really **short-term forecasts**

Conclusion

Potential issues with ARIMA in statsmodels and otherwise

1. Subjectivity in picking optimal p,q,d
2. Bad for long-term forecasts
3. Requires huge amounts of data
4. Computationally time- and memory-consuming
5. SARIMA is the alternative model for seasonal time-series
6. More intuitive models such as exponential smoothing or Holt Winters method

Another use of ARIMA




```
predicted=562.940747, expected=569.000000
predicted=567.126009, expected=583.679993
predicted=586.658874, expected=585.799988
predicted=582.087659, expected=594.690002
predicted=585.576938, expected=603.840027
predicted=603.348320, expected=606.469971
predicted=610.338663, expected=598.159973
predicted=600.153310, expected=598.570007
predicted=594.958368, expected=584.890015
predicted=585.237264, expected=578.169983
predicted=578.867483, expected=584.299988
predicted=567.346313, expected=587.849976
predicted=587.895157, expected=586.789978
predicted=586.769776, expected=578.309998
predicted=569.097284, expected=579.690002
predicted=584.372810, expected=590.789978
predicted=598.993654, expected=592.500000
```